



**Politecnico
di Torino**



Advanced Machine Learning

A.A. 2023/2024

Tatiana Tommasi

One step backward: Naïve Bayes Classifier

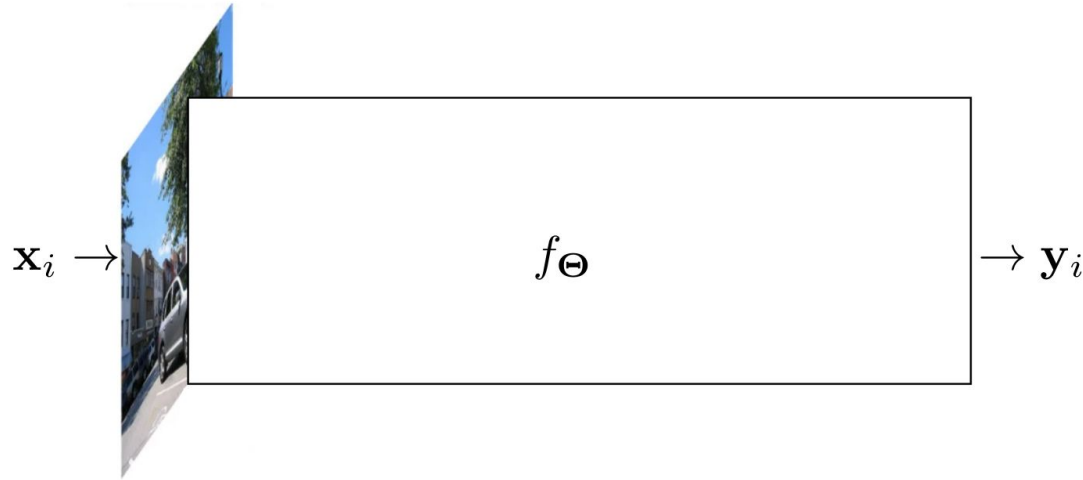
Given:

- Class prior $P(Y)$
- d conditionally independent features $X_1, \dots, X_d$ given the class label Y
- For each X_i feature, we have the conditional likelihood $P(X_i|Y)$

Naïve Bayes Decision rule:

$$\begin{aligned} f_{NB}(\mathbf{x}) &= \arg \max_y P(x_1, \dots, x_d | y) P(y) \\ &= \arg \max_y \prod_{i=1}^d P(x_i | y) P(y) \end{aligned}$$

One step forward: Deep Learning



How good are our predictions?

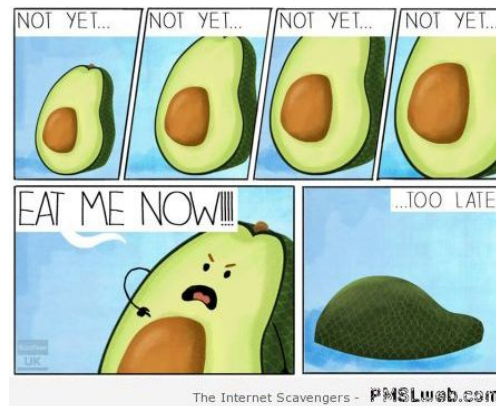
The avocado problem:

Example:

- Imagine to buy your first avocado
- How do you understand if it is ripe?
- Gather data: buy avocados and open them
- Features:
 - color: from dark green to dark brown
 - softness: from rock hard to mushy
- What is your prediction strategy on new avocados?
- Is perfect prediction of avocados possible?



<https://www.istockphoto.com/>



The Internet Scavengers - PMSLab.com

How good are our predictions?

The avocado problem:

Example:

- Imagine to buy your first avocado
- How do you understand if it is ripe?
- Gather data: buy avocados and open them
- Features:
 - color: from dark green to dark brown
 - softness: from rock hard to mushy
- What is your prediction strategy on new avocados?
- Is perfect prediction of avocados possible?



<https://www.istockphoto.com/>



Training Data



Build a
Model



Make predictions
on a **new** sample



How good are our predictions?

The avocado problem:

Example:

- Imagine to buy your first avocado
- How do you understand if it is ripe?
- Gather data: buy avocados and open them
- Features:
 - color: from dark green to dark brown
 - softness: from rock hard to mushy
- What is your prediction strategy on new avocados?
- Is perfect prediction of avocados possible?



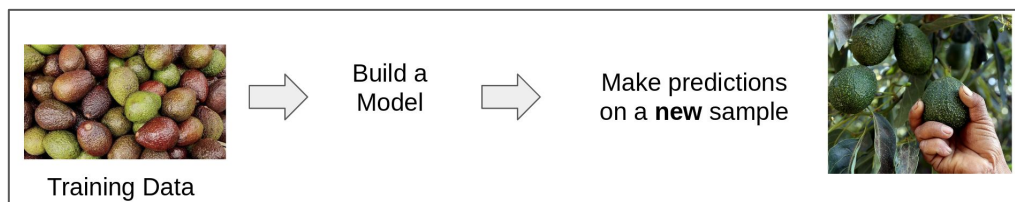
<https://www.istockphoto.com/>

...why do we minimize the training error while we care about the test error?

...is it always possible to get 100% accuracy?

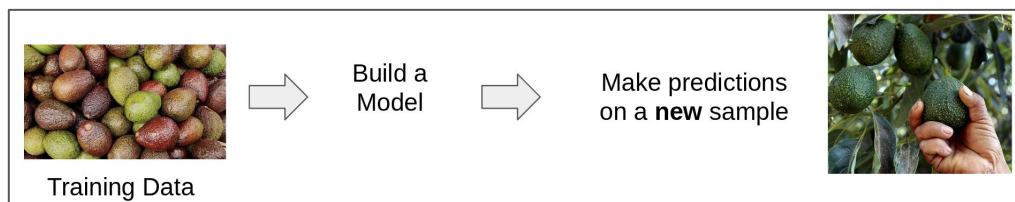
...what are the important parameters characterizing a learning problem?

Notation



- $\mathcal{X}$ is the input space, and $\mathcal{Y}$ is the output space
- In classification problems, output space $\mathcal{Y}$ comprised of K possible classes (or categories)
- For simplicity, we'll just call them $\mathcal{Y} := \{1, 2, \dots, K\}$
(When $K = 2$, we typically use $\mathcal{Y} = \{0, 1\}$ or $\mathcal{Y} = \{-1, +1\}$)
- Labeled example: $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$. Interpretation:
 - $\mathbf{x}$ represents the description or measurements of an object
 - y is the category to which that object belongs

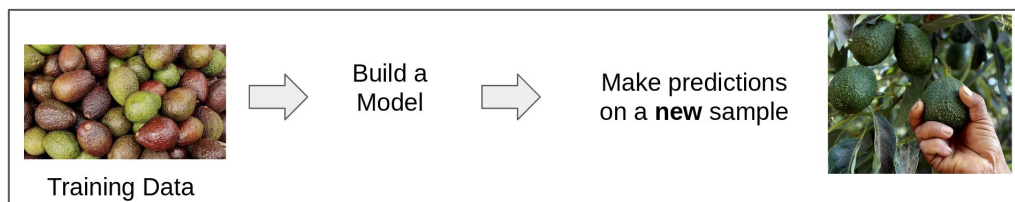
Loss



- Assume there's a distribution $\mathcal{D}$ over space of labeled examples $\mathcal{X} \times \mathcal{Y}$.
- $\mathcal{D}$ is unknown (e.g., we don't know its functional form), but it represents the population we care about.
- Assume also that the labels are produced by a *correct* labeling function $f : \mathcal{X} \rightarrow \mathcal{Y}$ and that $Y = f(X)$
- We define the **loss** of the classifier h as the function that measures the error between the assigned prediction and the prediction of the correct labeling function

$$L(h(X), f(X)) = L(h(X), Y) = \begin{cases} 1, & \text{if } h(X) \neq Y \\ 0, & \text{if } h(X) = Y \end{cases}$$

True Risk



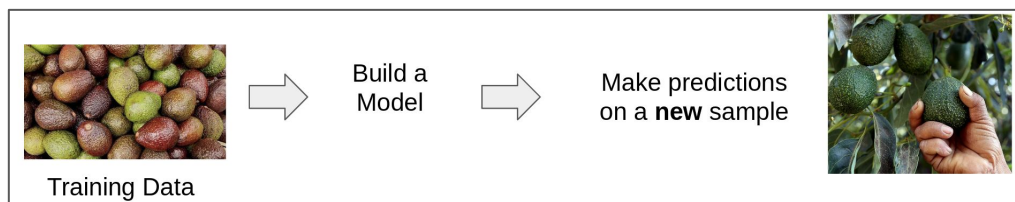
- We define the **loss** $L(h(X), Y)$ of the classifier h as the function that measures the error between the assigned prediction and the prediction of the correct labeling function
- The **true risk** of a classifier h is the probability that it does not predict the correct label on a random data point drawn from $\mathcal{D}$

$$R_{\mathcal{D}}(h) := \mathbb{P}[h(X) \neq f(X)] = \mathbb{P}[h(X) \neq Y]$$

- In other words, the **true risk** of the learner h is the expected value of the loss

$$R_{\mathcal{D}}(h) := \mathbb{E}_{X \sim \mathcal{D}}\{L(h(X), Y)\}$$

True Risk



- We define the **loss** $L(h(X), Y)$ of the classifier h as the function that measures the error between the assigned prediction and the prediction of the correct labeling function
- The **true risk** of a classifier h is the probability that it does not predict the correct label on a random data point drawn from $\mathcal{D}$

$$R_{\mathcal{D}}(h) := \mathbb{P}[h(X) \neq f(X)] = \mathbb{P}[h(X) \neq Y]$$

- In other words, the **true risk** of the learner h is the expected value of the loss

$$R_{\mathcal{D}}(h) := \mathbb{E}_{X \sim \mathcal{D}}\{L(h(X), Y)\}$$

$$GenLoss(h) = \sum_{(X,Y) \in \mathcal{D}} L(h(X), Y)P(X, Y)$$

Bayes Classifier

- Assume $\mathcal{X}$ is discrete for now
- Suppose $(\mathbf{x}, y) \sim \mathcal{D}$
- Want to find $h : \mathcal{X} \rightarrow \mathcal{Y}$ that minimizes
$$\mathbb{P}[h(\mathbf{x}) \neq y] = \sum_{\mathbf{x} \in \mathcal{X}} \mathbb{P}[h(X) \neq y | X = \mathbf{x}] \cdot \mathbb{P}[X = \mathbf{x}]$$
- For every $\mathbf{x} \in \mathcal{X}$, set $h(\mathbf{x})$ to minimize $\mathbb{P}[h(X) \neq Y | X = \mathbf{x}]$
- $\mathbb{P}[h(X) \neq Y | X = \mathbf{x}] = 1 - \mathbb{P}[h(X) = Y | X = \mathbf{x}]$
- So, to minimize $\mathbb{P}[h(X) \neq Y | X = \mathbf{x}]$, maximize $\mathbb{P}[h(X) = Y | X = \mathbf{x}]$
- i.e. choose

$$h(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \mathbb{P}[Y = y | X = \mathbf{x}]$$

As we already know, this is the **Bayes classifier**

Bayes Risk

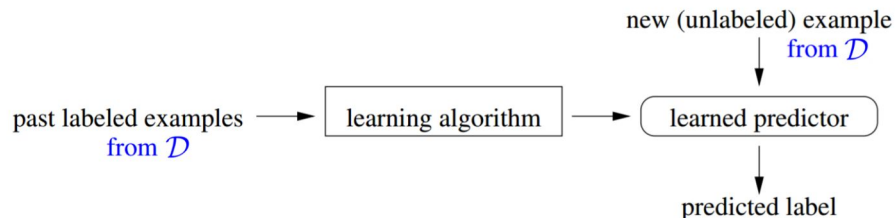
- Denote by h^* the Bayes classifier
- For the **Bayes Risk** $R_{\mathcal{D}}(h^*)$ it holds $R_{\mathcal{D}}(h^*) \leq R_{\mathcal{D}}(h)$ for any $\mathcal{H}$ and $h \in \mathcal{H}$
 - You cannot improve over the Bayes Risk, regardless of what ML algorithm you use!
- How do I know if I reached the Bayes Risk?
Andrew Ng's tip at NIPS'16: Estimate the Bayes Risk with the human performance

Batch Learning

- In practical cases we do not know f and we need to estimate h from the data
- The learner's input:
 - Training data, $S = \{(\mathbf{x}_1, y_1) \cdots (\mathbf{x}_m, y_m)\} \in (\mathcal{X} \times \mathcal{Y})^m$
- The learner's output:
 - A prediction rule, $h : \mathcal{X} \rightarrow \mathcal{Y}$
- What should be the goal of the learner?
Intuitively, h should be correct on future examples

Batch Learning - IID condition

- When is there any hope for finding a classifier with high accuracy?
- Key assumption: Data $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$ are *identically and independently distributed (i.i.d.)* random labeled examples with distribution $\mathcal{D}$, i.e., an i.i.d. sample from $\mathcal{D}$.



This assumption is the connection between what we've seen in the past to what we expect to see in the future

What does an accurate classifier look like?

Empirical Risk

- Training data $S = \{(\mathbf{x}_1, y_1) \cdots (\mathbf{x}_m, y_m)\} \in (\mathcal{X} \times \mathcal{Y})^m$ is an i.i.d. sample from some fixed but unknown probability distribution $\mathcal{D}$ over space of labeled examples $\mathcal{X} \times \mathcal{Y}$
- A learning algorithm takes S as input and returns a predictor $h : \mathcal{X} \rightarrow \mathcal{Y}$
- Calculate the **empirical risk** by benchmarking the prediction against ground truth:

$$R_S(h) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}[h(\mathbf{x}_i) \neq y_i]$$

- In this setting, can any learning algorithm always provide a non-trivial guarantee on the error of the predictor it returns?

No: some assumptions/conditions are required (“No free lunch” theorem)

Can Only Be Probably Correct

- **Claim:** Even when $R_{\mathcal{D}}(h^*) = 0$, we can't hope to find h s.t. $R_{\mathcal{D}}(h) = 0$
- **Proof:** for every $\epsilon \in (0, 1)$ take $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2\}$ and $\mathbb{P}_{\mathcal{D}}(\{\mathbf{x}_1\}) = 1 - \epsilon$, $\mathbb{P}_{\mathcal{D}}(\{\mathbf{x}_2\}) = \epsilon$
- The probability not to see $\mathbf{x}_2$ at all among m i.i.d. examples is $(1 - \epsilon)^m \approx e^{-\epsilon m}$
- So, if $\epsilon \ll 1/m$ we're likely not to see $\mathbf{x}_2$ at all, and we will not know its label!
- **Relaxation:** We'd be happy with $R_{\mathcal{D}}(h) \leq \epsilon$, where ϵ is user-specified

Can Only Be Approximately Correct

- Recall that the input to the learner is randomly generated
- There's always a (very small) chance to see the same example again and again
- **Claim:** No algorithm can guarantee $R_{\mathcal{D}}(h) \leq \epsilon$ for *sure*
- **Relaxation:** We'd allow the algorithm to fail with probability δ , where $\delta \in (0, 1)$ is user-specified
 - The probability is over the random choice of examples

Probably Approximately Correct (PAC) Learning

- The learner doesn't know $\mathcal{D}$ and the Bayes predictor.
- The learner receives accuracy parameter ϵ and confidence parameter δ
- The learner can ask for training data S containing $m(\epsilon, \delta)$ examples (that is, the number of examples can depend on the value of ϵ and δ but it can't depend on $\mathcal{D}$ or the Bayes function)
- Learner should output a hypothesis h s.t. $\mathbb{P}[R_{\mathcal{D}}(h) \leq \epsilon] \geq 1 - \delta$
- That is, the learner should be Probably (with probability at least $1 - \delta$) Approximately (up to accuracy ϵ) Correct: **PAC Learning**

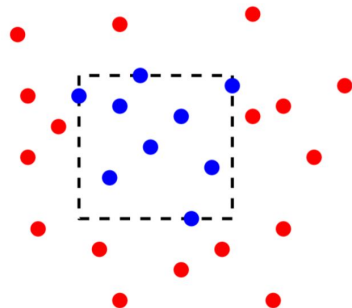
The setup of PAC Learning was introduced by L. Valiant (1984). He won the Turing Award for it in 2010.

Realizability Assumption

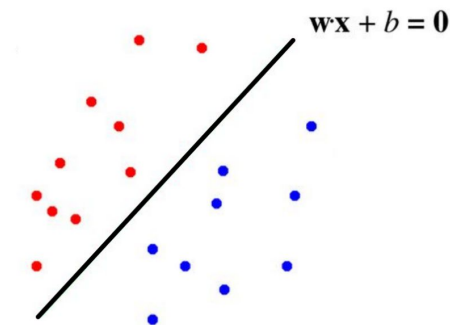
- Consider binary classification with $\mathcal{Y} = \{0, 1\}$.
- **Realizability assumption:** Assume that, for a given class $\mathcal{H}$ of functions from $\mathcal{X} \rightarrow \{0, 1\}$, there exists $h^* \in \mathcal{H}$ such that $R_{\mathcal{D}}(h^*) = 0$
 - This implies that h^* is the Bayes classifier
- The learner knows $\mathcal{H}$

Examples of function classes $\mathcal{H}$:

Rectangles in $\mathbb{R}^2$
(Blue dots = positive examples)



Linear classifiers in $\mathcal{X} = \mathbb{R}^d$



Learning in the Realizable Setting

What is a sensible learning algorithm for the realizable setting?

- Given training data S with m samples, return any $h \in \mathcal{H}$ such that $R_S(h) = 0$.
- This is a special case of **Empirical Risk Minimization** (ERM), that is

$$h_S = \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \mathbb{1}[h(\mathbf{x}_i) \neq y_i]$$

- We can show that for the consistent classifier $h \in \mathcal{H}$ obtained from the data by searching for the minimal empirical risk (ERM: $R_S(h) = 0$), the true risk $R_{\mathcal{D}}(h)$ remains bounded

PAC Learning

Definition (PAC learnability)

A hypothesis class $\mathcal{H}$ is PAC learnable if there exists a function $m_{\mathcal{H}} : (0, 1)^2 \rightarrow \mathbb{N}$ and a learning algorithm with the following property:

- for every $\epsilon, \delta \in (0, 1)$
- for every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$ such that the realizability assumption holds in $\mathcal{H}$

when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$, the algorithm returns a hypothesis h such that, with probability of at least $1 - \delta$ (over the choice of the examples), $R_{\mathcal{D}}(h) \leq \epsilon$

$m_{\mathcal{H}}$ is called the **sample complexity** of learning $\mathcal{H}$ PAC learnability formalizes the question: Can I learn anything with this hypothesis class (under the realizability assumption)? And how many samples I need?

What is Learnable and How to Learn?

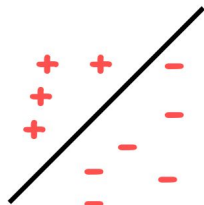
- Let $\delta \in (0, 1)$, $\epsilon > 0$, and $m \geq \frac{\ln(|\mathcal{H}|/\delta)}{\epsilon}$. Then, with probability $1 - \delta$, we have $R_{\mathcal{D}}(h) \leq \epsilon$

Clearly these bounds are useful if $\ln |\mathcal{H}|$ is finite and not too large w.r.t. $|S|$

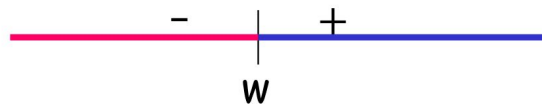
- Does PAC learning makes sense without the realizability assumption?
- What about infinite hypothesis classes?

Infinite Hypothesis Classes?

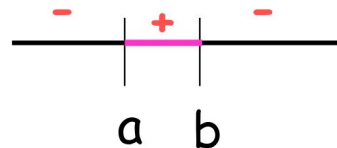
E.g., linear separators in $\mathbb{R}^d$



E.g., thresholds on the real line



E.g., intervals on the real line



Infinite Hypothesis Classes?

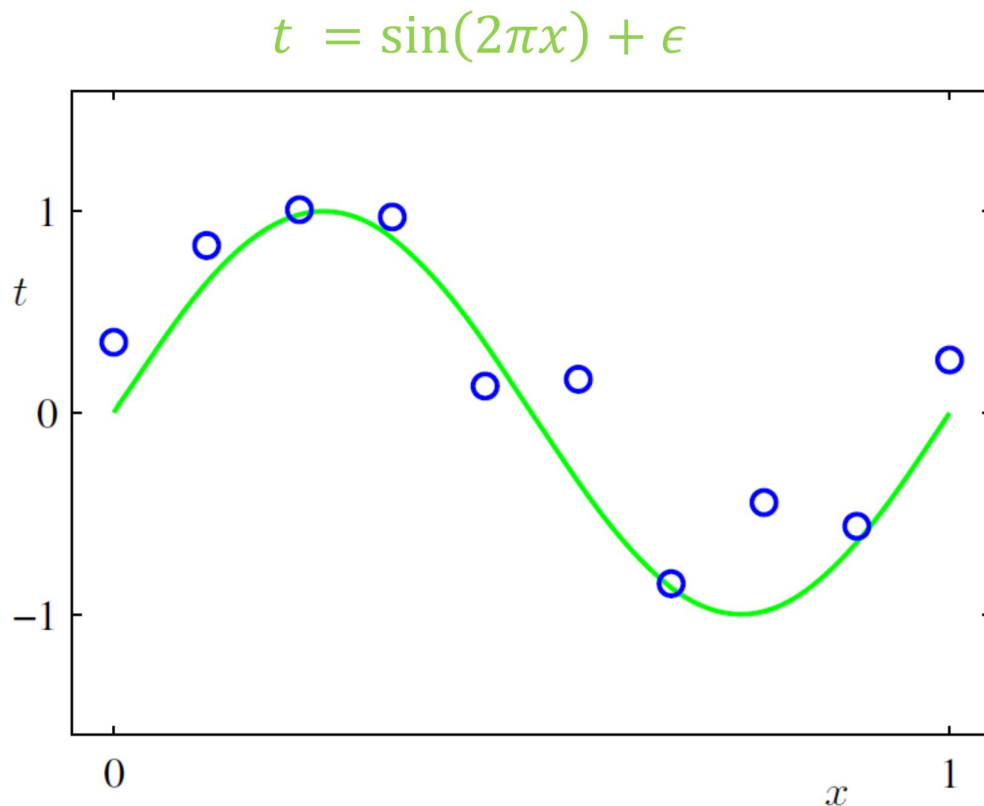
Discretization trick

- In practice, any learning algorithm we use will be implemented on a computer
- This means that any parametrization of the algorithm, e.g. the coefficient in a linear classifier, are stored in a floating point representation
- Hence, say, 32 bits will be used for each parameter, giving 2^{32} possibilities
- So, the class is actually finite!
- So with d parameters we have $|\mathcal{H}_{discrete}| \leq 2^{32d}$ and $\ln |\mathcal{H}_{discrete}| \leq 10d$
 - The famous rule of thumb of taking a number of samples at least 10 times the number of parameters!

Things can go wrong

Example: regression using polynomial curve

$$\begin{aligned}y(x, \mathbf{w}) &= \\&= w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \\&= \sum_{j=0}^M w_jx^j\end{aligned}$$



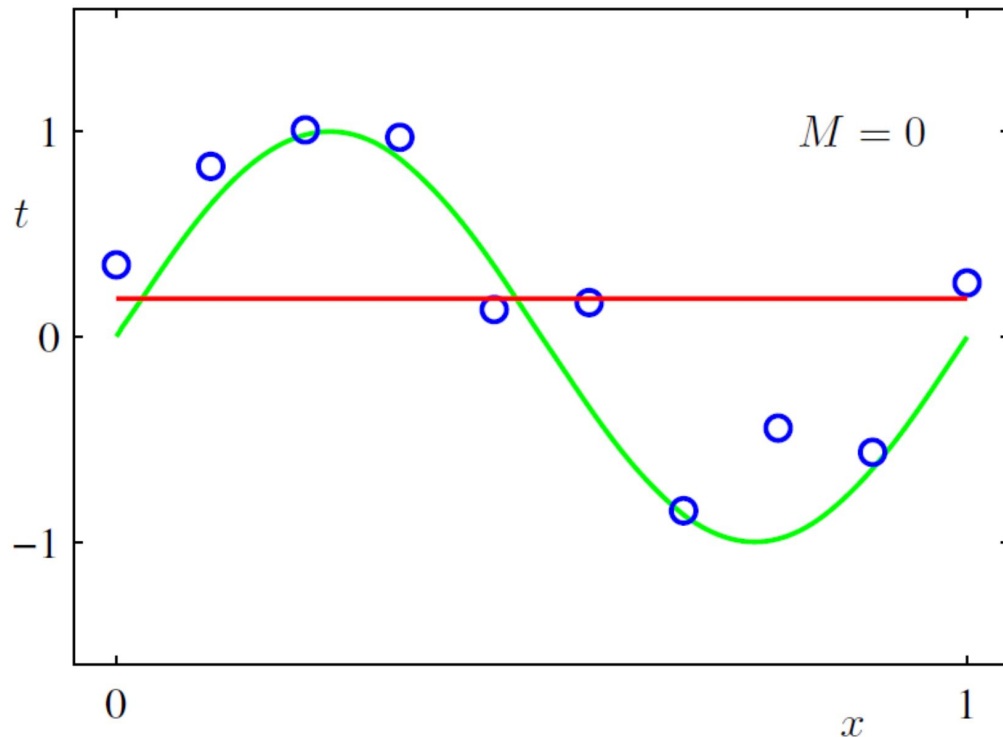
Things can go wrong

Example: regression using polynomial curve

$$\begin{aligned}y(x, \mathbf{w}) &= \\&= w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \\&= \sum_{j=0}^M w_jx^j\end{aligned}$$



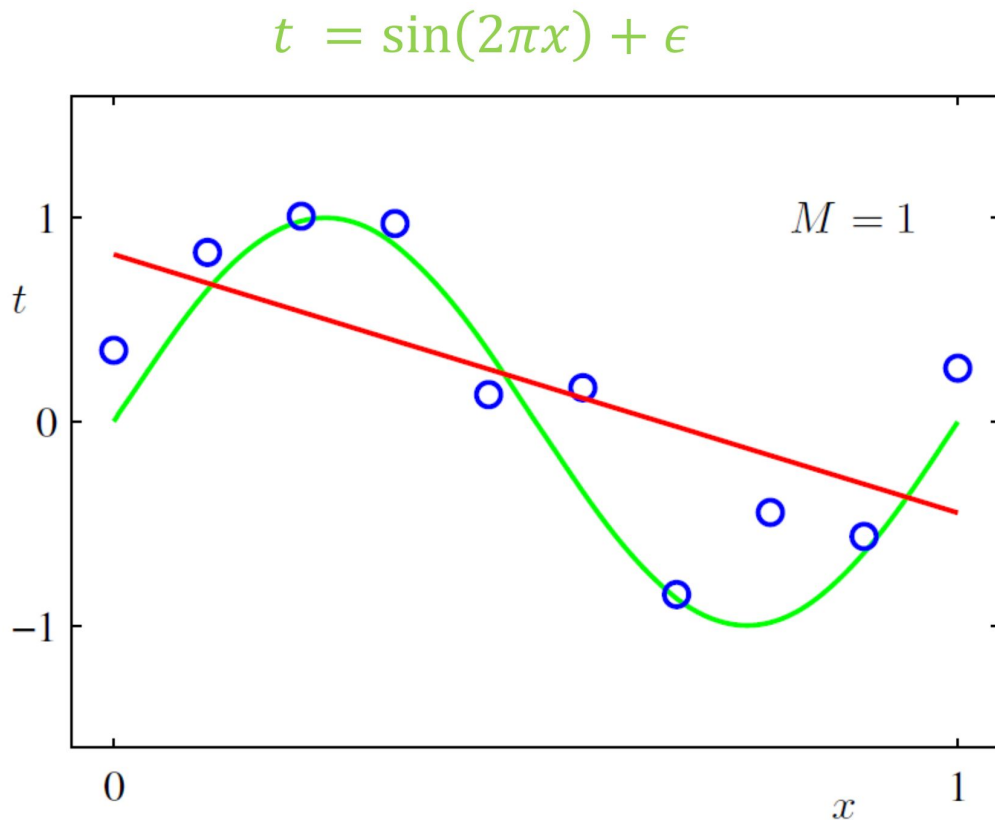
$$t = \sin(2\pi x) + \epsilon$$



Things can go wrong

Example: regression using polynomial curve

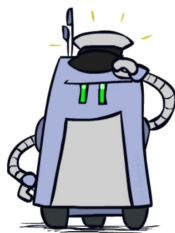
$$\begin{aligned}y(x, \mathbf{w}) &= \\&= w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \\&= \sum_{j=0}^M w_jx^j\end{aligned}$$



Things can go wrong

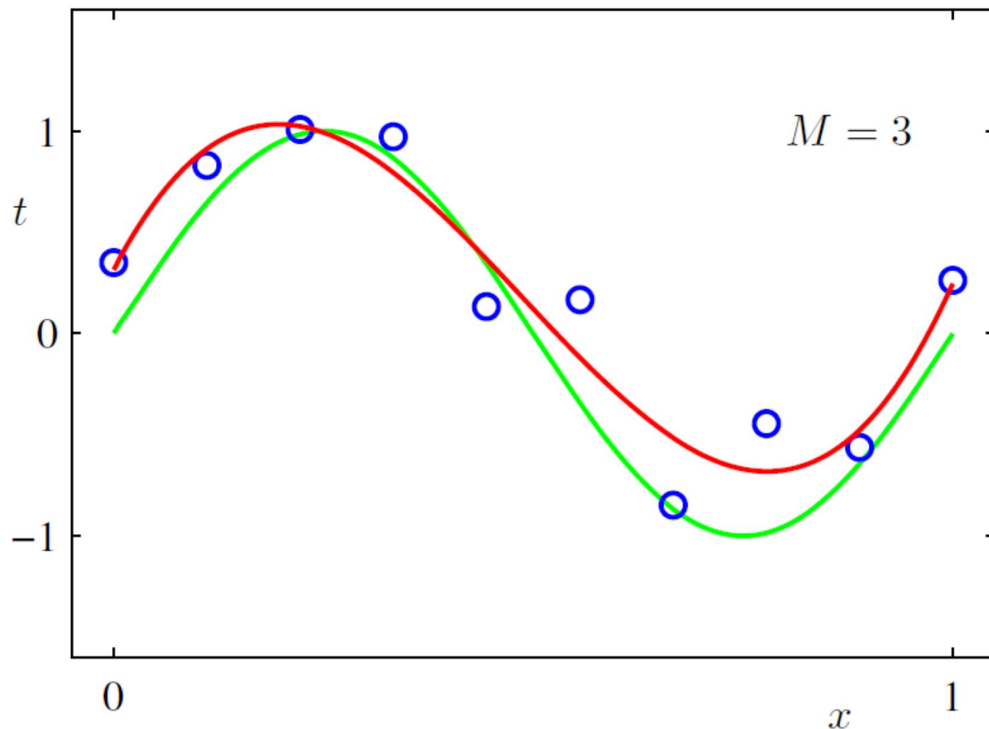
Example: regression using polynomial curve

$$\begin{aligned}y(x, \mathbf{w}) &= \\&= w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \\&= \sum_{j=0}^M w_jx^j\end{aligned}$$



<http://www-inst.eecs.berkeley.edu/~cs188/fa19/>

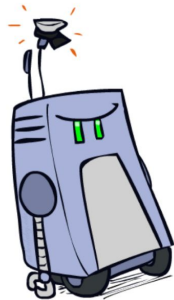
$$t = \sin(2\pi x) + \epsilon$$



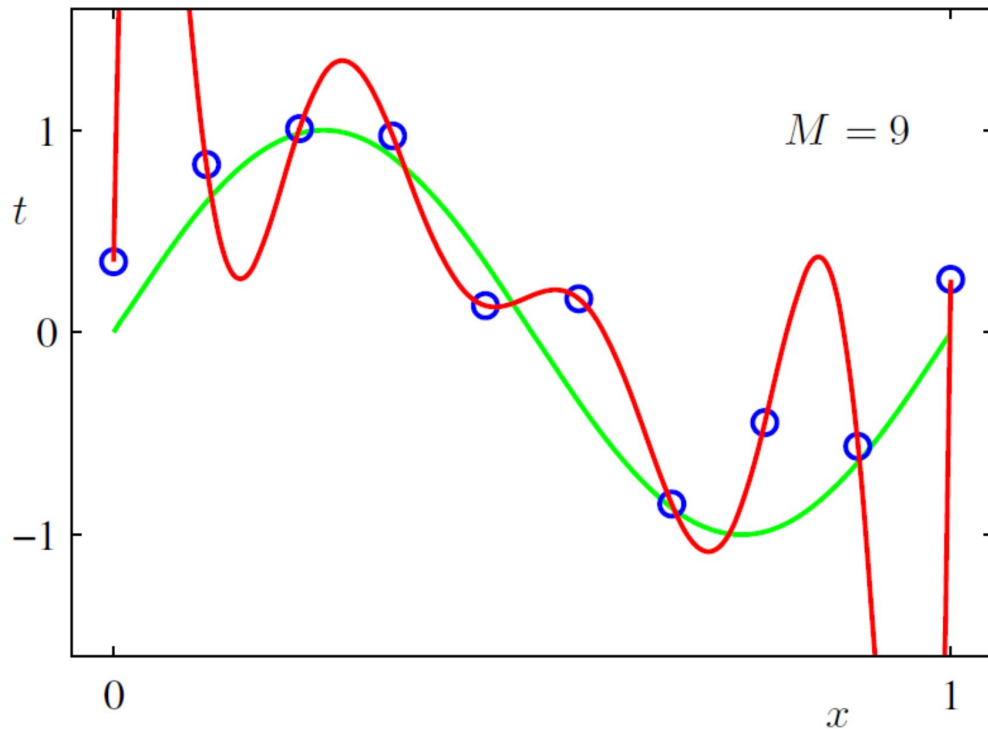
Things can go wrong

Example: regression using polynomial curve

$$\begin{aligned}y(x, \mathbf{w}) &= \\&= w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \\&= \sum_{j=0}^M w_jx^j\end{aligned}$$

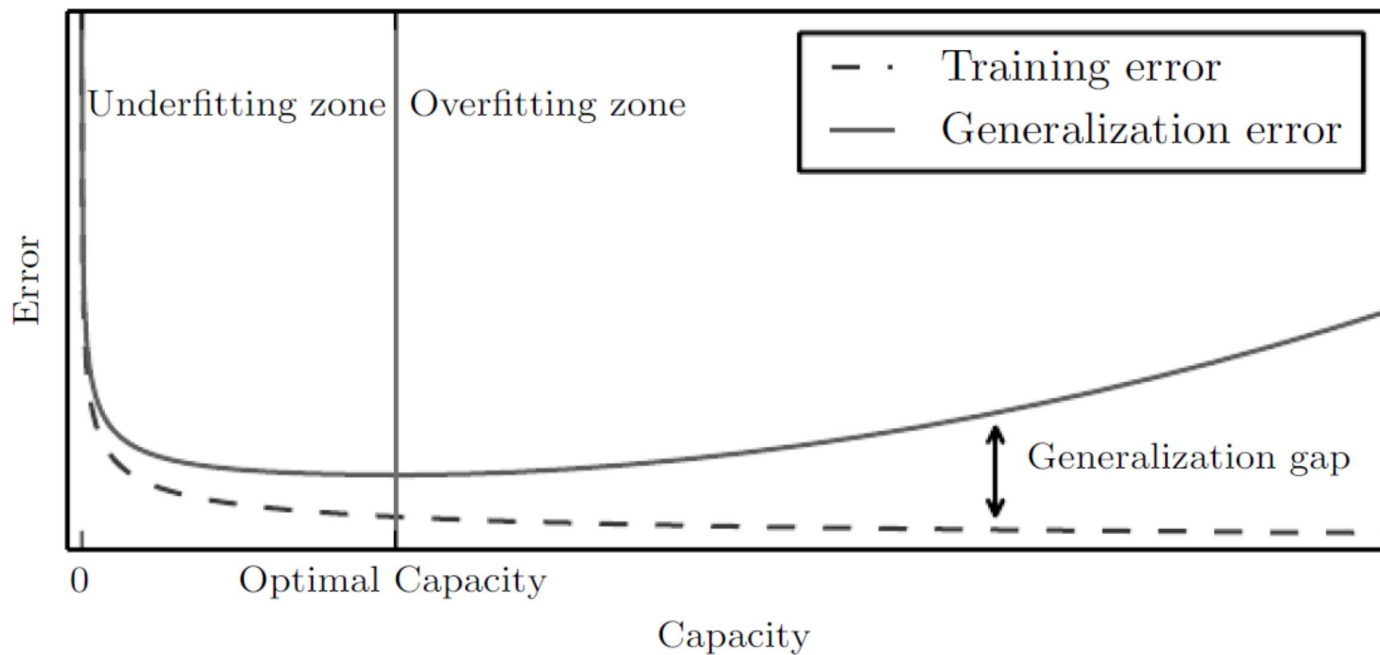


$$t = \sin(2\pi x) + \epsilon$$



Things can go wrong

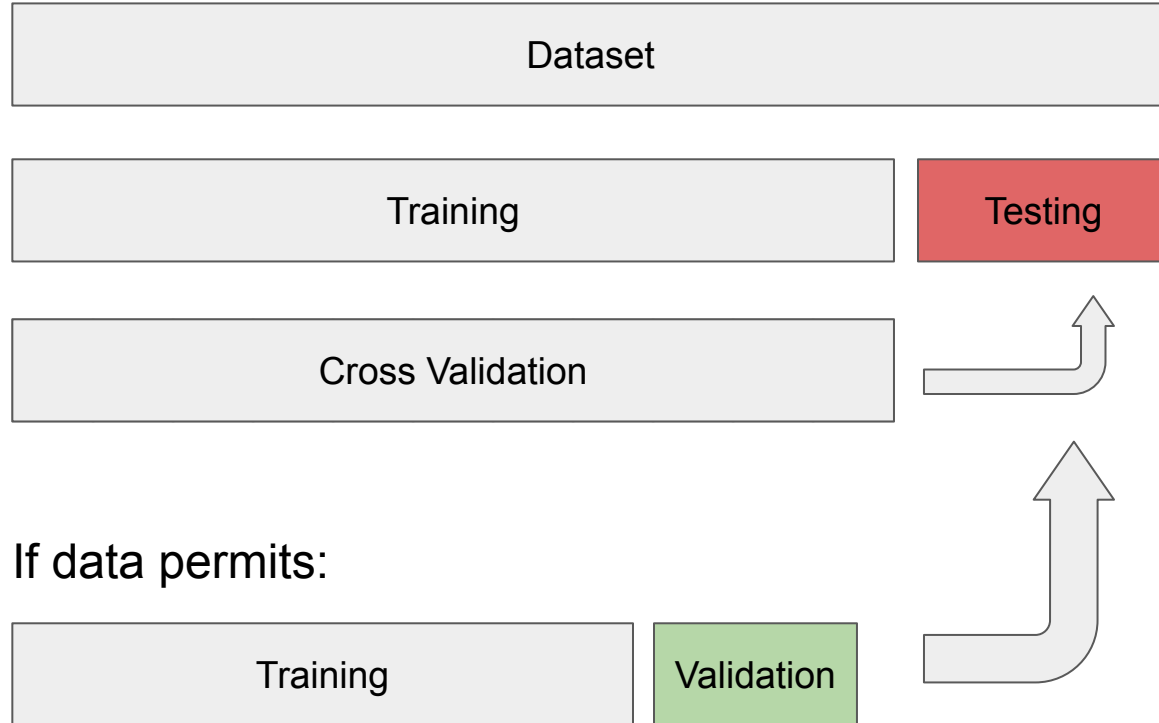
General
Phenomenon:



Overfitting

- Definition: When predictor has excellent performance on the training set, but its performance on the true “world” is very poor
- We already know some cases when overfitting does not happen: finite hypothesis classes and enough samples
- Hence, overfitting is not about noise nor about having training error 0, but about the fact that training error and true error are two different things!
- Only under certain conditions, they behaves similarly, i.e., with enough samples wrt the complexity of your hypothesis class

Cross Validation



k-fold Cross Validation

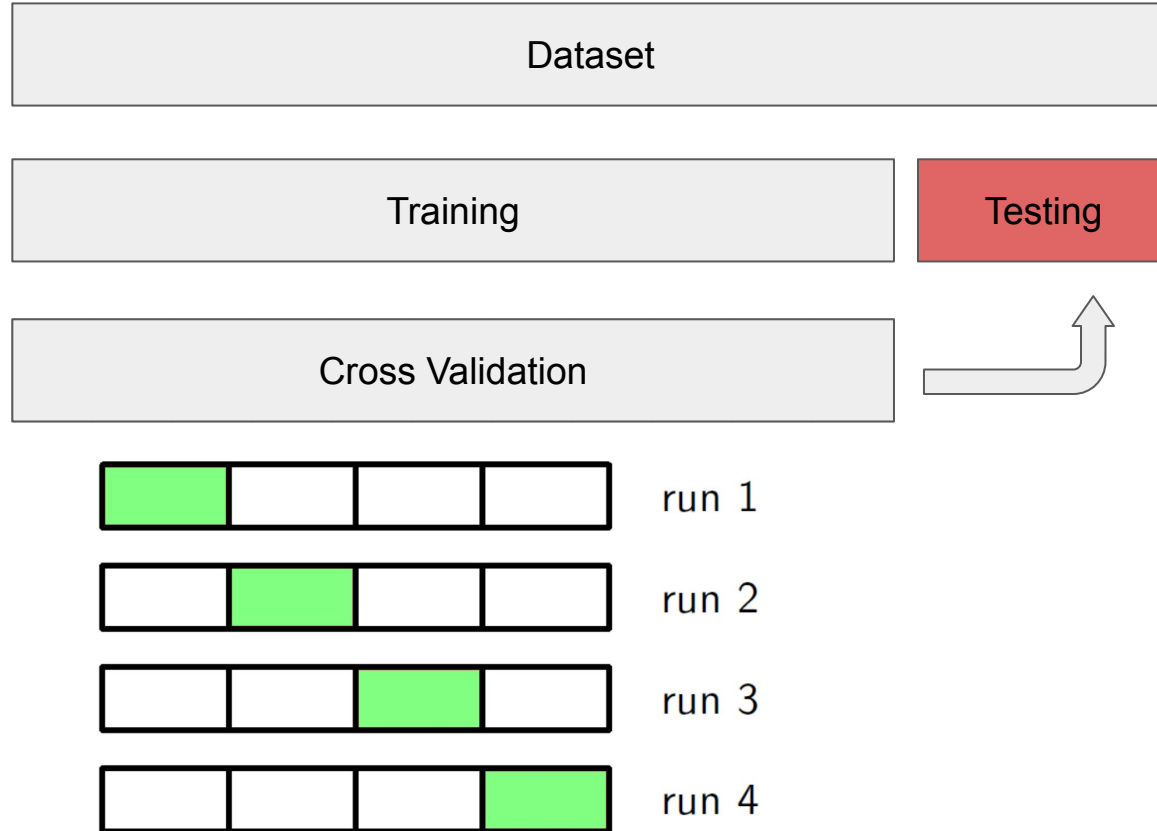
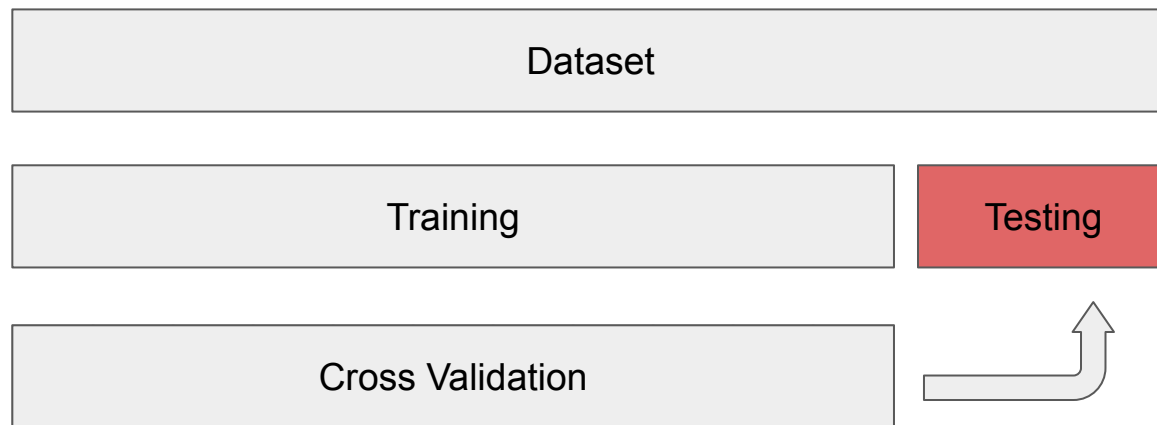


Figure from Machine Learning and Pattern Recognition, Bishop

k-fold Cross Validation

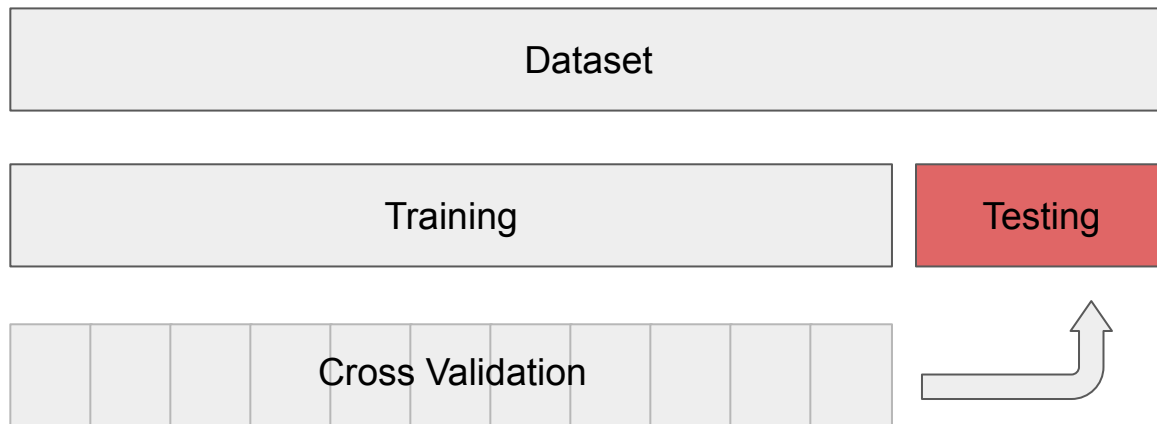


Algorithm 1 k-Fold Cross Validation for Model Selection

- 1: Input: training set $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$
learning algorithm A and a set of parameter values Θ
 - 2: partition S into $S_1, S_2, \dots, S_k$
 - 3: **for** Each $\theta \in \Theta$ **do**
 - 4: **for** $i = 1 \dots k$ **do**
 - 5: $h_{i,\theta} = A(S \setminus S_i; \theta)$
 - 6: **end for**
 - 7: $error(\theta) = \frac{1}{k} \sum_{i=1}^k L_{S_i}(h_{i,\theta})$
 - 8: **end for**
 - 9: Output: $\theta^* = \arg \min_{\theta} [error(\theta)]$; $h_{\theta^*} = A(S; \theta^*)$
-

Figure from Machine Learning and Pattern Recognition, Bishop

k-fold Cross Validation



If $k = |S|$
Leave-One-Out
Cross Validation



Algorithm 1 k-Fold Cross Validation for Model Selection

- 1: Input: training set $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$
learning algorithm A and a set of parameter values Θ
- 2: partition S into $S_1, S_2, \dots, S_k$
- 3: **for** Each $\theta \in \Theta$ **do**
- 4: **for** $i = 1 \dots k$ **do**
- 5: $h_{i,\theta} = A(S \setminus S_i; \theta)$
- 6: **end for**
- 7: $error(\theta) = \frac{1}{k} \sum_{i=1}^k L_{S_i}(h_{i,\theta})$
- 8: **end for**
- 9: Output: $\theta^* = \arg \min_{\theta} [error(\theta)]$; $h_{\theta^*} = A(S; \theta^*)$

Train - Validation - Test

In practice, we usually have one pool of examples and we split them into three sets:

- Training set: apply the learning algorithm with different parameters on the training set to produce $H = h_1, \dots, h_r$
- Validation set: Choose h^* from H based on the validation set
- Test set: Estimate the true error of h^* using the test set

Note: You cannot avoid the use of a test set, the error on the validation set is negatively biased!

- **If the number of samples in the test set is too small, I cannot reliably estimate the true error!**
- If my gain over the baseline is too small, I might not be able to exclude the case that the gain is just due to a random fluctuation

Model Selection In Summary

- There is an *unavoidable* trade-off between complexity of the model and its variance
- More powerful models approximate better the Bayes function, but need more data
- Never use the test set for any selection of parameters/algorithms

Model Selection In Summary

- There is an *unavoidable* trade-off between complexity of the model and its variance
- More powerful models approximate better the Bayes function, but need more data
- Never use the test set for any selection of parameters/algorithms



<https://en.ephoto360.com/blood-writing-text-online-77.html>

Learning Solution to Overfitting

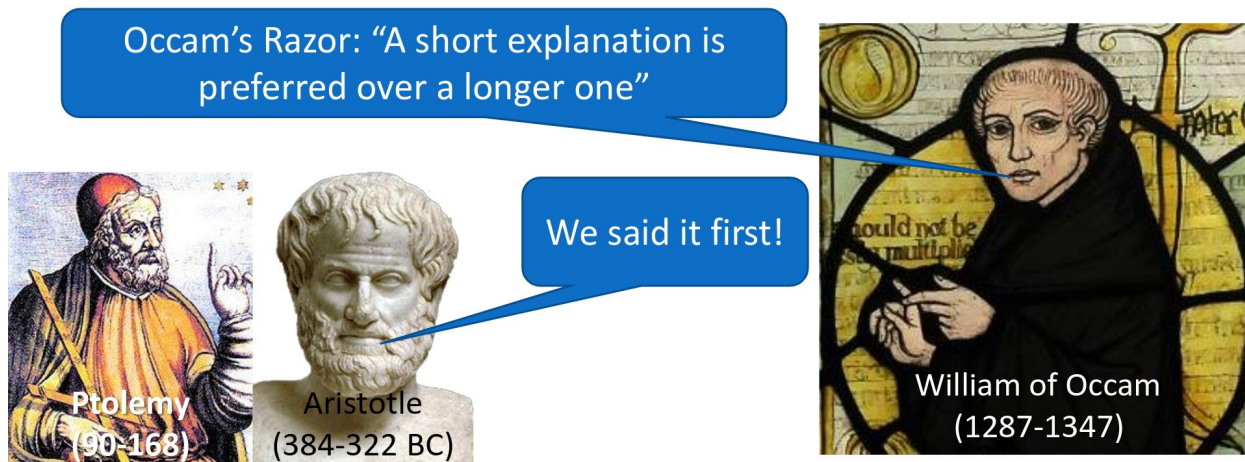
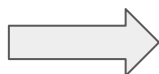


Figure Credit: Nati Srebro



Control complexity by penalizing complex models in learning: **regularization**

Regularization

- Given training data S with m samples, return any $h \in \mathcal{H}$ such that $R_S(h) = 0$.
- This is a special case of **Empirical Risk Minimization** (ERM), that is

$$h_S = \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \mathbb{1}[h(\mathbf{x}_i) \neq y_i]$$

Regularization

- Given training data S with m samples, return any $h \in \mathcal{H}$ such that $R_S(h) = 0$.
- This is a special case of **Empirical Risk Minimization** (ERM), that is

$$h_S = \arg \min_{h \in \mathcal{H}} \left[\frac{1}{m} \sum_{i=1}^m \mathbb{1}[h(x_i) \neq y_i] + \lambda \text{ Complexity}(h) \right]$$

- λ is a parameter, a positive number that serves as a conversion rate between the loss and the hypothesis complexity (they might not have the same scale)
- the form of the complexity /regularization function depends on the hypothesis space
- we still need cross validation and in this case we need to include the parameter λ - select the one that gives the best validation score

Linear Regression

Linear Regression

- So far, mostly focused on classification problems, where the label space $\mathcal{Y} = \{1, 2, \dots, K\}$ is discrete.
- **Regression** analysis considers prediction problems where label space $\mathcal{Y}$ is continuous (e.g. $[0, 1]$, $\mathbb{R}^d$, etc.)
- Measuring quality of a predictor $h : \mathcal{X} \rightarrow \mathcal{Y}$ using prediction error, $\mathbb{P}[h(X) \neq Y]$ doesn't make much sense in the regression context (why?)
- **Goal:** find $h : \mathcal{X} \rightarrow \mathcal{Y}$, from some function class $\mathcal{H}$, minimizing error measured using a loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$, i.e.

$$\mathbb{E}[L(Y, h(X))]$$

Flashback: Loss Function

- A loss function $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ maps decisions to costs: $L(\hat{y}, y)$ defines the penalty paid for predicting $\hat{y}$ when the true value is y .
- Standard choice for classification: 0/1 loss

$$L_{0/1}(\hat{y}, y) = \begin{cases} 0 & \hat{y} = y \\ 1 & \text{otherwise} \end{cases}$$

- Standard choice for regression: squared loss $L(\hat{y}, y) = (\hat{y} - y)^2$
 - Not the only possible choice

Empirical Loss

- We consider a parametric function $h_{\mathbf{w}}(\mathbf{x})$
E.g., linear function: $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$
- The empirical loss of function $y = h_{\mathbf{w}}(\mathbf{x})$ on a set S :

$$L_S(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N L(h_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

- We will find an ERM over $\mathbb{R}^d$ w.r.t. L
 - E.g., Least Squares minimizes the empirical loss for the squared loss.

Linear Predictors in 1d

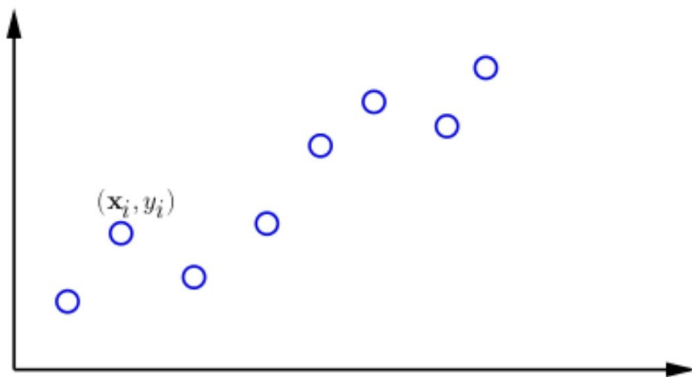
- Our features are vectors in $\mathbb{R}$, i.e. $\mathcal{X} = \mathbb{R}$
- The labels are real numbers, i.e. $\mathcal{Y} = \mathbb{R}$
- Consider predictors of the form

$$\hat{y} = h_{(w,b)}(x) = w x + b$$

- Hypothesis class: $\mathcal{H} = \{h_{(w,b)} : w, b \in \mathbb{R}\}$
 - Infinite, but remember the discretization trick!

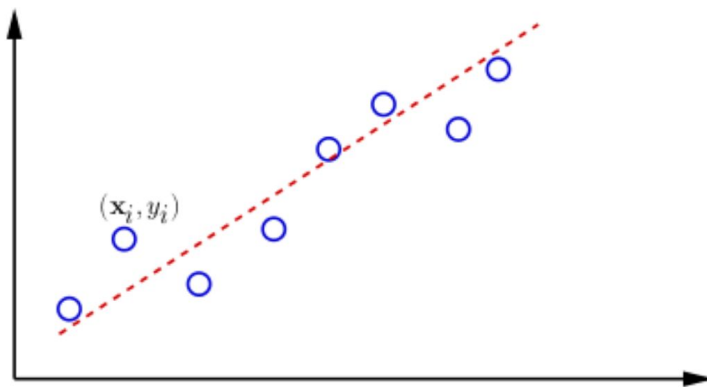
Linear Fitting to Data

- We want to fit a linear function to an observed set of points $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]$ with associated labels $\mathcal{Y} = [y_1, \dots, y_N]$.
 - Once we fit the function, we want to use it to *predict* the y for new $\mathbf{x}$.
- Least squares (LSQ) fitting criterion: find the function that minimizes sum (or average) of square distances between actual y_i in the training set and predicted ones, that is ERM



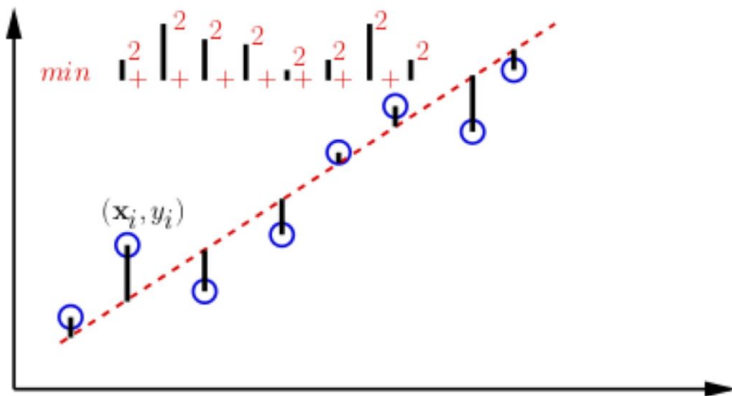
Linear Fitting to Data

- We want to fit a linear function to an observed set of points $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]$ with associated labels $\mathcal{Y} = [y_1, \dots, y_N]$.
 - Once we fit the function, we want to use it to *predict* the y for new $\mathbf{x}$.
- Least squares (LSQ) fitting criterion: find the function that minimizes sum (or average) of square distances between actual y_i in the training set and predicted ones, that is ERM



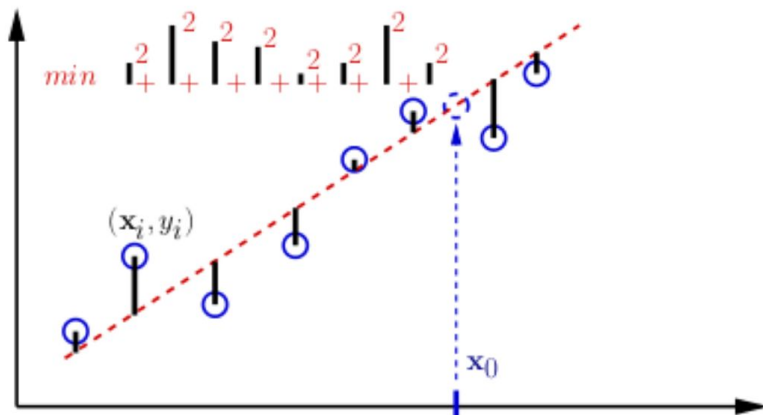
Linear Fitting to Data

- We want to fit a linear function to an observed set of points $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]$ with associated labels $\mathcal{Y} = [y_1, \dots, y_N]$.
 - Once we fit the function, we want to use it to *predict* the y for new $\mathbf{x}$.
- Least squares (LSQ) fitting criterion: find the function that minimizes sum (or average) of square distances between actual y_i in the training set and predicted ones, that is ERM



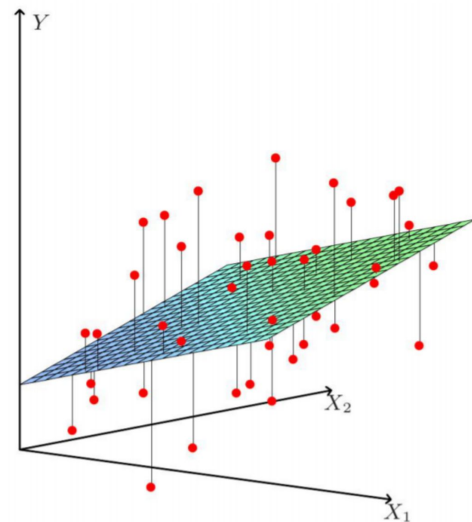
Linear Fitting to Data

- We want to fit a linear function to an observed set of points $\mathcal{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]$ with associated labels $\mathcal{Y} = [y_1, \dots, y_N]$.
 - Once we fit the function, we want to use it to *predict* the y for new $\mathbf{x}$.
- Least squares (LSQ) fitting criterion: find the function that minimizes sum (or average) of square distances between actual y_i in the training set and predicted ones, that is ERM



Linear Functions

- General form: $\hat{y} = h_{(b, \mathbf{w})}(\mathbf{x}) = b + w_1 x_1 + \cdots + w_d x_d = b + \langle \mathbf{w}, \mathbf{x} \rangle$
- 1D case ($\mathcal{X} = \mathbb{R}$): a line
- $\mathcal{X} = \mathbb{R}^2$: a plane
- Hyperplane in general, d -dimensional case
- Hypothesis class: $\mathcal{H} = \{h_{(b, \mathbf{w})} : b \in \mathbb{R}, \mathbf{w} \in \mathbb{R}^d\}$



Least Squares Criterion

- Given pairs of input/output values, find $(b, \mathbf{w})$ to minimize the prediction errors (on average)
- I.e. using square loss on $\mathcal{H} = \{h_{(b, \mathbf{w})} : b \in \mathbb{R}, \mathbf{w} \in \mathbb{R}^d\}$: squared error on $(\mathbf{x}, y)$ is $(y - (b + \langle \mathbf{w}, \mathbf{x} \rangle))^2$

Least Squares in Matrix / Vector Form

Given training data

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_N^\top & - \end{bmatrix} \in \mathbb{R}^{N \times d} \quad \mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N,$$

find $b \in \mathbb{R}$ and $\mathbf{w} \in \mathbb{R}^d$ to minimize

$$\sum_{i=1}^N (y_i - (b + \langle \mathbf{w}, \mathbf{x}_i \rangle))^2 = \left\| \mathbf{y} - \begin{bmatrix} \mathbf{1} & \mathbf{X} \end{bmatrix} \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \right\|_2^2$$

Simplification: Replace $\mathbf{X}$ with $\begin{bmatrix} \mathbf{1} & \mathbf{X} \end{bmatrix}$ and $\mathbf{w}$ with $\begin{bmatrix} b \\ \mathbf{w} \end{bmatrix}$, to have the simpler expression

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

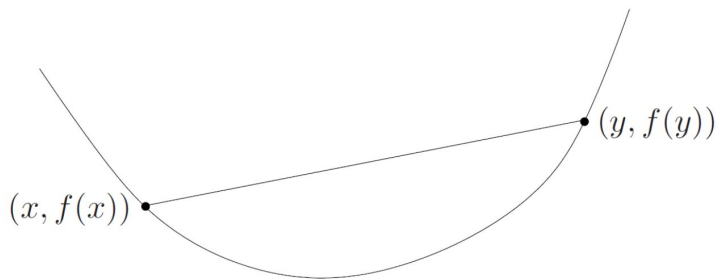
Least Squares via Calculus

- Least squares criterion is convex function of $\mathbf{w}$; so suffices to find $\mathbf{w}$ where gradient is zero

Jensen's inequality:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$$

for all x, y and $\alpha \in [0, 1]$



Let us further assume that f is a **differentiable** function, so that we can compute its **derivative** $\frac{df}{dx}$ at all points x .

Intuition tells us that the minimizer x is where $\frac{df(x)}{dx} = 0$.

Slide Credit: E. Rodolà

Least Squares via Calculus

- Least squares criterion is convex function of $\mathbf{w}$; so suffices to find $\mathbf{w}$ where gradient is zero
- Take gradient with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = 2\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

- This is zero when

$$(\mathbf{X}^\top \mathbf{X})\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

This is a linear system of equations in $\mathbf{w}$

- If $\mathbf{X}^\top \mathbf{X}$ is invertible, the solution is

$$\mathbf{w}_{\text{ols}} := (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

“ordinary least squares”

Linear Predictor in 1d

$$\arg \min_w \sum_i (y_i - wx_i)^2 \quad \text{forget about the b for one second....}$$

We just take the derivative w.r.t. to w and set to 0:

$$\frac{\partial}{\partial w} \sum_i (y_i - wx_i)^2 = 2 \sum_i -x_i (y_i - wx_i) \Rightarrow$$

$$2 \sum_i x_i (y_i - wx_i) = 0 \Rightarrow 2 \sum_i x_i y_i - 2 \sum_i wx_i x_i = 0$$

$$\sum_i x_i y_i = \sum_i wx_i^2 \Rightarrow$$

$$w = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

add the bias term

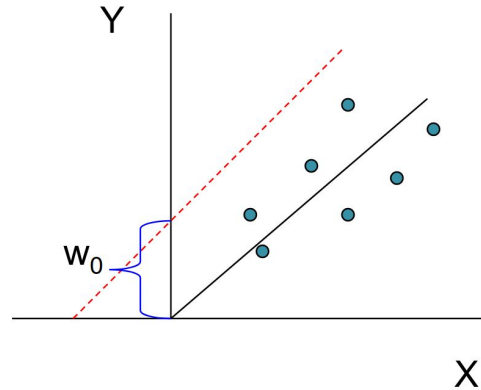
- So far we assumed that the line passes through the origin
- What if the line does not?
- No problem, simply change the model to

$$y = w_0 + w_1x$$

- Can use least squares to determine w_0 , w_1

$$w_0 = \frac{\sum_i y_i - w_1 x_i}{n}$$

$$w_1 = \frac{\sum_i x_i (y_i - w_0)}{\sum_i x_i^2}$$



Add regularization (Ridge Regression)

$$\frac{1}{2} \|y - Xw\|_2^2 + \frac{\alpha}{2} \|w\|_2^2$$

- Now our objective consists in minimizing both the loss function and a regularization term.
- The regularizer prevents the model to become too complex
- It is a standard way to control overfitting and becomes particularly important for small number of samples (N) and large vector dimensionality (d)

$$\begin{aligned}\nabla_w \|y - Xw\|_2^2 + \frac{\alpha}{2} \|w\|_2^2 \\ = X^\top (Xw - y) + \alpha w = 0\end{aligned}$$

$$\Rightarrow w = (X^\top X + \alpha I)^{-1} X^\top y \quad \text{Ridge Regression}$$

Beyond Linear Models: Polynomial Regression

- We can easily learn *non-linear* predictors in the same way
- Consider $\mathcal{X} = \mathbb{R}$, and the predictor

$$\hat{y} = f(x) = b + w_1x + w_2x^2 + \cdots + w_mx^m$$

- Is this a linear model? No! Yet it is linear in $(b, w_1, \cdots, w_m)$
- It can be learned with Least Square, as

$$\hat{y} = h(x) = \langle \phi(x), w \rangle = b + w_1x + w_2x^2 + \cdots + w_mx^m$$

where $\phi(x) = [1, x, x^2, x^3, \cdots, x^m] \in \mathbb{R}^{m+1}$

- The same trick can be used when $\mathcal{X} = \mathbb{R}^d$.

Fitting Polynomials

- Transform $\mathbf{x} \in \mathbb{R}^d$ into a vector $\phi(\mathbf{x}) \in \mathbb{R}^m$
- $\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = \sum_{i=1}^m w_i \phi_i(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$, still linear
- Examples:
 - $\phi_i(\mathbf{x}) = \exp(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_i\|^2}{s_i^2})$, Isotropic Gaussians
- For example, for LS we have $\sum_{i=1}^N (\mathbf{w}^\top \phi(\mathbf{x}_i) - y_i)^2 = \|\mathbf{y} - \boldsymbol{\Phi} \mathbf{w}\|_2^2$ where

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_1(\mathbf{x}_1) & \phi_2(\mathbf{x}_1) & \cdots & \phi_m(\mathbf{x}_1) \\ \phi_1(\mathbf{x}_2) & \phi_2(\mathbf{x}_2) & \cdots & \phi_m(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(\mathbf{x}_N) & \phi_2(\mathbf{x}_N) & \cdots & \phi_m(\mathbf{x}_N) \end{bmatrix}$$

- We solve it exactly as before!

Summary

- Learning can only be Probably and Approximately Correct (PAC)
 - true risk vs empirical risk
 - loss
 - empirical risk minimization can lead to learning algorithms with reasonable generalization guarantees (within some conditions)
- Underfitting / Overfitting
 - cross-validation
- Linear Regression
 - ERM with square loss over linear predictors is efficient: just invert a matrix
 - LS learning with basis functions (e.g. polynomials) is still linear
 - without and with regularization